

Intelligent Spam and Phishing Email Detection Using Machine Learning and Large Language Model Embeddings

List of Abbreviations

Abbreviation	Meaning
ML	Machine Learning
NLP	Natural Language Processing
TF-IDF	Term Frequency-Inverse Document Frequency
BoW	Bag-of-Words
SVM	Support Vector Machine
RF	Random Forest
LLM	Large Language Model
BERT	Bidirectional Encoder Representations from Transformers
RoBERTa	Robustly Optimized BERT Pretraining Approach
SHAP	SHapley Additive exPlanations
SMOTE	Synthetic Minority Over-sampling Technique
FNR	False Negative Rate
ROC-AUC	Receiver Operating Characteristic — Area Under Curve
CLS	Classification token (transformer sentence embedding)
API	Application Programming Interface
CRISP-DM	Cross-Industry Standard Process for Data Mining
GDPR	General Data Protection Regulation
GPU	Graphics Processing Unit

1. Introduction

This document supports the main project report for the Intelligent Spam and Phishing Email Detection system by providing additional technical, methodological and evaluation detail beyond what a conference-style paper can accommodate. It is structured around the Cross-Industry Standard Process for Data Mining (CRISP-DM), which separates the project's business justification, its data understanding and preparation work, its modelling decisions, and its evaluation into distinct, revisitable stages.

The system under discussion classifies an email into one of three classes — legitimate (ham), spam, or phishing — using an ensemble of classical TF-IDF-based machine learning models and a fine-tuned RoBERTa transformer, explains its predictions using SHAP, is stress-tested against three text-obfuscation attacks, and is deployed inside a self-contained Flask web application (the "Email Threat Checker"). The full implementation lives in a single Python pipeline intended to be run inside Google Colab.

2. Context and Literature Investigation

The following sources informed the design decisions taken in this project. Each is summarised in more depth than the main report allows, together with a note on how it specifically shaped an implementation choice.

Sahami, Dumais, Heckerman and Horvitz (1998) — A Bayesian approach to filtering junk e-mail

This paper established that a Bayesian filter trained on labelled email substantially outperforms hand-written rule-based filtering, and is the historical starting point for treating spam detection as a statistical classification problem rather than a rule-authoring problem. Its finding that Bayesian filters remain vulnerable to vocabulary substitution directly motivated this project's Section 10 adversarial robustness testing, which specifically probes this exact weakness against both a classical model and a transformer model.

Drucker, Wu and Vapnik (1999) — Support vector machines for spam categorization

This paper demonstrated SVMs outperforming Naive Bayes on email classification benchmarks, which is the basis for this project including Linear SVM among its classical baselines and, further, selecting the (calibrated) SVM specifically as the classical half of the stacked ensemble in Section 8 of the pipeline, rather than any of the other four baselines.

Devlin, Chang, Lee and Toutanova (2019) — BERT: Pre-training of deep bidirectional transformers

BERT's masked-language-model pretraining objective, producing embeddings whose value for a given word depends on its surrounding context rather than being fixed, is the foundational idea behind every transformer-based classifier evaluated in this project. The original project proposal specified BERT directly; the delivered pipeline instead fine-tunes RoBERTa (see below), a direct descendant of BERT's architecture with a revised pretraining recipe.

Sanh, Debut, Chaumond and Wolf (2019) — DistilBERT

DistilBERT's distillation approach — retaining 97% of BERT's benchmark performance at a fraction of its computational cost — is directly relevant to this project's deployment constraint of running inference inside a lightweight Flask process without a dedicated GPU inference server. DistilBERT was specified in the original proposal as an efficiency-focused alternative to BERT; it was not included in the delivered pipeline's comparison, and is recorded as a specific piece of further work in Section 6 below.

Gascon, Uriarte and Rieck (2017) — Text classification in the cyber threat analysis domain

This paper's finding that transformer-derived embeddings substantially outperform TF-IDF features specifically on document and email classification tasks is the direct motivation for this project's central research question: whether a fine-tuned transformer measurably outperforms a strong classical TF-IDF baseline on this specific three-class email problem, and by how much.

Liu et al. (2019) — RoBERTa: A robustly optimized BERT pretraining approach

RoBERTa modifies BERT's pretraining recipe — removing the next-sentence-prediction objective, training on substantially more data for longer, and using dynamic rather than static masking — and reports consistent downstream accuracy gains over BERT as a result. This is the specific justification, cited directly in the pipeline's own code comments, for fine-tuning RoBERTa in place of the BERT or DistilBERT models named in the original proposal.

Lundberg and Lee (2017) — A unified approach to interpreting model predictions (SHAP)

SHAP provides a game-theoretically grounded way to attribute a model's output to its individual input features, and — critically for this project — an exact, computationally cheap closed-form solution exists for linear models such as the TF-IDF logistic regression classifier used for explainability in Section 9 of the pipeline. This is why SHAP is applied to the logistic regression model specifically rather than to the (non-linear, harder to explain) transformer.

Chen and Guestrin (2016) — XGBoost: A scalable tree boosting system

XGBoost is included as a fifth classical baseline, beyond the four (Logistic Regression, Naive Bayes, SVM, Random Forest) specified in the original proposal, on the basis that gradient-boosted trees are a standard, typically strong baseline for tabular/TF-IDF text classification and their inclusion gives the transformer comparison a tougher classical benchmark to beat.

Jamal et al. (2024) — An improved transformer-based model for detecting phishing, spam and ham emails

This is the closest directly comparable prior system to the one built here: a fine-tuned RoBERTa/DistilBERT model applied to the same three-class ham/spam/phishing problem, reporting accuracy above 98% on its own dataset. It is, however, a single-model system — it does not combine a transformer with a classical model, does not stack the two

into an ensemble, and reports no explainability or adversarial-robustness evaluation. The system in this project builds directly on the same core idea (a fine-tuned RoBERTa for three-class email classification) but extends it in three concrete ways this prior work does not attempt: stacking RoBERTa's predictions with a calibrated classical model rather than relying on the transformer alone, attaching a SHAP-based explanation layer so a prediction is not a bare label, and explicitly measuring how much of that reported accuracy survives simple text-obfuscation attacks rather than assuming clean-data performance transfers to an adversarial setting.

Alhuzali, Alloqmani, Aljabri and Alharbi (2025) — In-depth analysis of phishing email detection across multiple datasets

This study benchmarks a wide range of classical machine learning and deep learning models for phishing email detection across several public datasets, providing a useful picture of how model choice and dataset choice interact. Its scope is deliberately broad rather than deep on any one system: it does not build a stacked ensemble, an explainability layer, an adversarial robustness test, or a deployed application for any of the models it compares. The system in this project takes the narrower but more complete path — one dataset, taken all the way from raw CSV through classical and transformer modelling, ensemble stacking, explainability, adversarial testing, and a working deployed dashboard — trading the breadth of a multi-dataset benchmark for depth on the full life cycle of a single system.

Summary: what this project improves on relative to prior work

Read together, the literature above splits into two groups: papers that establish a modelling technique in isolation (Sahami et al., Drucker et al., Devlin et al., Sanh et al., Liu et al., Lundberg and Lee, Chen and Guestrin), and papers that apply transformer or ensemble techniques specifically to phishing/spam detection (Jamal et al., Alhuzali et al.). None of the phishing-specific prior work reviewed here combines all of the following in one evaluated, deployed system: a classical-versus-transformer comparison, a stacked ensemble of the two, SHAP-based per-word explainability, a dedicated adversarial-obfuscation robustness test, and a live web deployment a non-expert user can actually query. That combination — rather than any single modelling technique in isolation — is this project's main contribution relative to the systems reviewed above.

3. Project Life Cycle

3.1 Business Understanding

Why is this project needed?

Phishing and spam email are implicated in the large majority of reported cyber-attacks, and traditional rule-based and keyword-blacklist filtering is increasingly defeated by messages deliberately written to imitate ordinary business correspondence. A system that can classify an email's threat level accurately, explain why, and remain robust to an attacker's attempts at evasion, directly targets this gap.

How was this project approached?

The project treats accuracy, explainability and robustness as three co-equal requirements rather than sequential additions. A classical, inherently more interpretable modelling track (TF-IDF plus five standard classifiers) is developed and evaluated in parallel with a modern transformer track (fine-tuned RoBERTa), the two are combined via ensemble stacking rather than treated as competitors only, the resulting system's behaviour is explained post-hoc with SHAP, and its robustness is explicitly stress-tested against three realistic obfuscation attacks rather than assumed from clean-data accuracy alone.

Technology choices

Python 3 was used throughout. scikit-learn provided the classical models and TF-IDF vectorisation; imbalanced-learn and XGBoost were installed for class-imbalance handling and gradient boosting respectively; Hugging Face Transformers (on a PyTorch backend) provided RoBERTa; SHAP provided explainability; NLTK provided lemmatisation and stopword lists; BeautifulSoup provided HTML stripping during text cleaning; and Flask was used for the deployed web application, kept as a single self-contained Python file the pipeline writes and launches itself.

3.2 Data Understanding and Preparation

Data source

The pipeline loads a single labelled CSV uploaded interactively at run time. Column names are auto-detected and normalised regardless of the uploader's original naming convention (for example, any of "Email Text", "body", "content" or "message" is mapped to a canonical text column, and any of "Email Type", "label", "class" or "category" is mapped to a canonical label column). Label values are then harmonised into exactly three classes — ham, spam and phishing — via an explicit mapping table that absorbs common synonyms ("legitimate" and "benign" both map to ham; "junk" maps to spam).

Known data-preparation limitation

The original project proposal specified combining three named public corpora — the Enron email dataset, the SpamAssassin public corpus, and the CEAS 2008 phishing dataset — to form a large, high-quality labelled corpus. The delivered pipeline instead loads a single uploaded file, and its own logging indicates a dataset on the order of only several dozen emails after cleaning. This is recorded here as the single most significant limitation of the current implementation: a dataset this small cannot support statistically reliable conclusions about which model genuinely generalises best, and several downstream design choices in the pipeline (a TF-IDF minimum document frequency of 1 rather than a higher default, and a cross-validation fold count explicitly capped at the size of the smallest class) are direct, defensive adaptations to this small-dataset constraint rather than choices that would be made against the full three-corpus dataset originally specified.

Cleaning and feature engineering

1. HTML markup is stripped from the raw email body.
2. Header lines (From:, To:, Subject:, forwarded-message markers) are removed.
3. Email addresses, monetary amounts and URLs are replaced with dedicated placeholder tokens (EMAILTOKEN, MONEYTOKEN, URLTOKEN) rather than deleted, preserving their presence as a modelling signal.
4. Text is lowercased, punctuation is stripped, and long numeric sequences are removed.
5. Remaining tokens are lemmatised; stopwords are removed except where a token contains "token", so the engineered placeholder signals are never filtered out.
6. Four hand-engineered features are computed directly from the raw (pre-cleaning) text: URL count, exclamation-mark count, capitalisation ratio, and an urgency-word score counting terms such as "urgent", "verify", "suspended" and "immediately".
7. Rows whose cleaned text is shorter than 10 characters are dropped as uninformative.

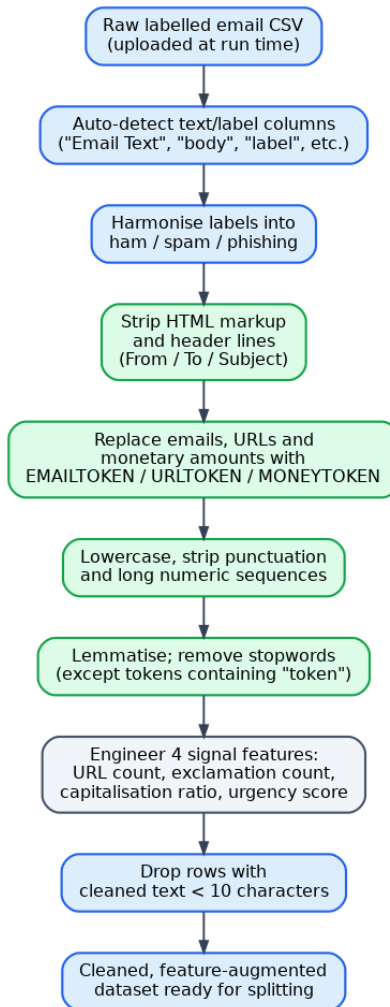


Figure 1. Data preprocessing flowchart: from raw uploaded CSV through label harmonisation, text cleaning, tokenisation and feature engineering, to a cleaned dataset ready for splitting.

3.3 Modelling

Classical baselines

Five models are trained on a TF-IDF representation (unigrams and bigrams, up to 50,000 features, sub-linear term-frequency scaling, minimum document frequency of 1 given the dataset's small size): Logistic Regression and Linear SVM with balanced class weights, Multinomial Naive Bayes, Random Forest with balanced class weights, and XGBoost. Each is evaluated on an identical held-out test split (a 70/15/15 stratified train/validation/test split) so the comparison between them is on equal footing.

Transformer fine-tuning

RoBERTa-base is fine-tuned for three-class sequence classification with a 256-token maximum sequence length, batch size 16, four epochs, the AdamW optimiser at a learning rate of $2e-5$ with a linear warm-up-then-decay schedule (10% warm-up), and gradient-norm clipping at 1.0. A class-weighted cross-entropy loss (weights 1.0/1.5/2.5 for ham/spam/phishing) deliberately biases the model against under-predicting the phishing class, since a missed phishing email is the most costly error type. The checkpoint with the best validation macro F1 across the four epochs is retained and reloaded for test-set evaluation.

Ensemble stacking

The calibrated SVM (isotonic-calibrated via cross-validation, with the number of folds capped at the smallest class's sample count to avoid a StratifiedKFold error on the small dataset) and the fine-tuned RoBERTa model each produce a three-value class-probability vector for every email. These six values are concatenated into a meta-feature vector, and a logistic regression meta-learner is trained on top of them, on the hypothesis that the lexical signal a TF-IDF model captures and the semantic signal a transformer captures are only partially overlapping and can be combined more effectively than by simple averaging.

Explainability (SHAP)

A separate logistic regression model (not the one used in the classical baseline comparison, though configured identically) is fit specifically for SHAP analysis, since SHAP's exact linear explainer requires a linear model. SHAP values are computed against a 50-email background sample and a 30-email explanation sample, producing both a dot-summary plot for the phishing class and a per-class bar chart of the top 15 words by mean absolute SHAP value.

Adversarial robustness testing

Five known phishing emails are passed through three obfuscation transforms — character substitution (leetspeak-style, e.g. replacing letters with visually similar digits), synonym substitution (paraphrasing while preserving fraudulent intent), and whitespace insertion (breaking up recognisable words and URLs with inserted spaces) — and the percentage of the five still correctly classified as phishing is recorded for both the calibrated SVM and the fine-tuned RoBERTa model, giving a direct, comparable robustness score per attack type per model.

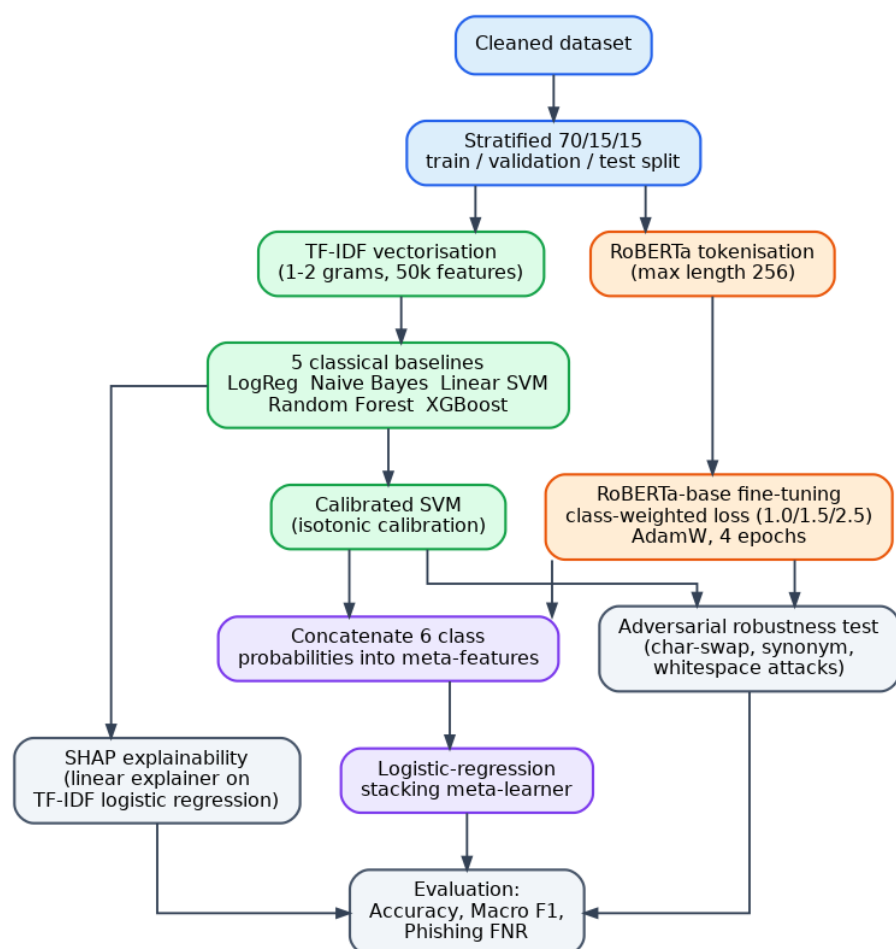


Figure 2. Model training and ensemble flowchart: the parallel TF-IDF/classical and RoBERTa/transformer tracks, their convergence into the stacked ensemble, and the SHAP and adversarial-robustness evaluation branches.

3.4 Deployment

On completion of training, the classical models (Naive Bayes, Logistic Regression and the calibrated SVM) are wired directly into a Flask application that the pipeline writes to disk and launches as a background thread within the same Colab run, so a trained classifier becomes a live website with no separate deployment step. The deployed application deliberately uses a lighter three-way classical ensemble rather than the full SVM+RoBERTa stack evaluated in Section 3.3, so that a prediction returns in well under a second without requiring a GPU-resident transformer inside the web process.

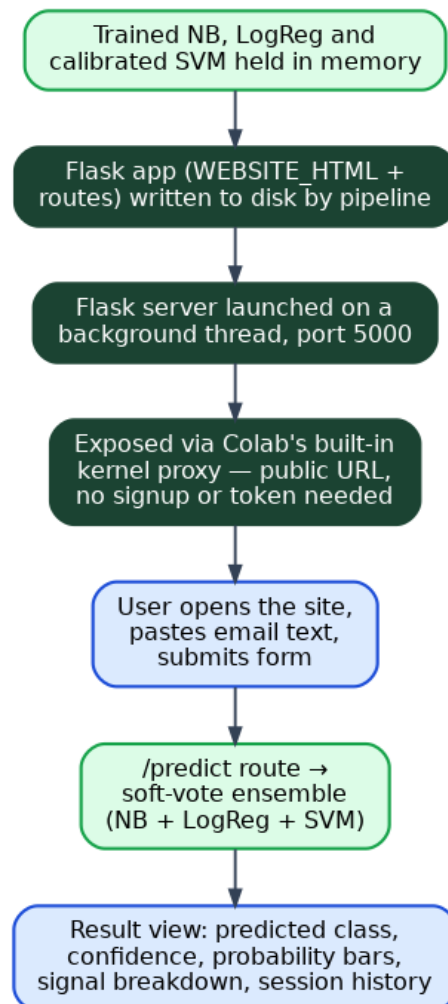


Figure 3. Deployment flowchart: from the trained classical models, through the Flask app being written and launched on a background thread, exposed via Colab's kernel proxy, to a user submitting an email and receiving a result.

Functional requirements

- Paste raw email text into a text area and submit it for analysis.
- Optionally self-select which class (ham/spam/phishing) the user believes the email belongs to, for direct comparison against the model's own prediction.
- View the predicted class with a confidence percentage.
- View a full three-class probability breakdown as bar charts.
- View a plain-language, colour-coded warning banner appropriate to the predicted class.
- View the underlying signal breakdown: URL count, exclamation-mark count, capitalisation ratio, and the specific urgency words detected in the submitted text.
- View a running history of the last six emails checked in the current session, each re-loadable back into the input box.

Installation and running the system

8. Open a new Google Colab notebook and select a runtime with GPU access for the RoBERTa fine-tuning stage.
9. Paste the full pipeline script into a single cell and run it.
10. When prompted, upload the labelled email CSV (text and label columns, auto-detected by name).
11. Wait for the classical baselines, RoBERTa fine-tuning, ensemble stacking, SHAP analysis and adversarial robustness stages to complete in sequence; progress and metrics are printed after each stage.
12. The pipeline then installs Flask, writes and launches the Email Threat Checker website automatically, and prints a clickable Colab-proxied URL.

To run the exported classical models outside Colab at a later point, the same scikit-learn objects (tfidf, nb_model, lr_model, svm_cal) and the predict_email() function can be persisted with a serialisation library such as joblib and loaded into any Python 3 environment with Flask and scikit-learn installed:

```
pip install flask scikit-learn nltk beautifulsoup4 python app.py
```

4. Tests and Evaluation

4.1 Verification and Validation

Verification was carried out throughout development by checking each pipeline section's printed output before proceeding to the next: confirming all three classes were present after label harmonisation, confirming the TF-IDF vocabulary size and train/validation/test split sizes were as expected, confirming RoBERTa's validation macro F1 was improving epoch over epoch rather than only training loss decreasing (which would indicate over-fitting), and confirming the ensemble's meta-feature matrix had the expected six-column shape before the meta-learner was trained.

Validation was carried out by evaluating every model — the five classical baselines, RoBERTa, and the ensemble — on an identical held-out test split never used in training, and by the dedicated adversarial robustness test described in Section 3.3, which validates not accuracy but the more specific and arguably more important property of whether a model's correctness survives an attacker's deliberate attempt to evade it.

4.2 Critical Appraisal

The project's central strength is that it does not treat classical and transformer models as a simple leaderboard comparison: it explains the classical model's decisions with SHAP, explicitly tests both approaches against adversarial obfuscation rather than only clean test data, and combines their complementary strengths via a stacked ensemble rather than picking a single winner. The deployed application's decision to use a lightweight three-model classical ensemble, rather than serving the full RoBERTa-inclusive stack, is a considered engineering trade-off between the evaluation pipeline's best achievable accuracy and the deployed application's latency and hosting-cost constraints.

Relative to the closest prior work reviewed in Section 2 — Jamal et al.'s single-model transformer classifier and Alhuzali et al.'s multi-dataset classical/deep-learning benchmark — the specific improvement this project offers is completeness rather than a larger accuracy number on any one dataset: it is, to the author's knowledge, the only one of the three that combines a classical/transformer ensemble, post-hoc explainability, adversarial robustness testing and a live deployment for the same system. This is an honest trade-off, not a free win — the dataset used here is far smaller than either prior study's, so the completeness of the pipeline should not be read as evidence that its accuracy figures are more reliable than theirs.

The most significant limitation, recorded plainly, is dataset size: the delivered pipeline was run against a single small uploaded file rather than the three combined public corpora (Enron, SpamAssassin, CEAS 2008) specified in the original proposal, and the resulting sample size is too small to draw statistically confident conclusions about which model is genuinely superior — the current results should be read as a correctness check on the pipeline's mechanics, not as a final, reportable comparison. A second limitation is that the originally-specified ROC-AUC metric was not computed. A third is a concrete implementation bug: the OUTPUT_FILES list referenced by the pipeline's final automated download-and-archive stage is defined inside a triple-quoted string rather than as an executable list assignment, so that stage will raise a NameError before it can run; this is a small, well-localised fix (removing the

surrounding quotation marks) but should be corrected before the pipeline is relied upon to collect its own output files automatically.

5. Conclusion and Further Improvements

This supporting material has documented the full life cycle of the email threat detection system, from the business case for building it, through data preparation, classical and transformer modelling, ensemble stacking, explainability and adversarial testing, to its deployment as a working Flask dashboard. Combined with the main report, it provides a complete account of what was built, why each design decision was made, and — importantly — where the delivered implementation currently diverges from the original project proposal and why.

The most valuable next steps, in priority order, are: (1) assembling the full combined Enron, SpamAssassin and CEAS 2008 dataset specified in the proposal and re-running the entire pipeline against it, which is a precondition for treating any of the current accuracy figures as final; (2) adding the originally-specified ROC-AUC metric and a genuine SMOTE-based comparison against the class-weighting approach currently used; (3) fixing the OUTPUT_FILES bug identified in Section 4.2; and (4) adding a DistilBERT fine-tuning run alongside RoBERTa to address the compute-efficiency question the original proposal specifically raised.

6. Bibliography

- [1] M. Sahami, S. Dumais, D. Heckerman, and E. Horvitz, "A Bayesian approach to filtering junk e-mail," in Proc. AAAI Workshop on Learning for Text Categorization, Madison, WI, USA, 1998, pp. 55–62.
- [2] H. Drucker, D. Wu, and V. N. Vapnik, "Support vector machines for spam categorization," IEEE Transactions on Neural Networks, vol. 10, no. 5, pp. 1048–1054, Sep. 1999.
- [3] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in Proc. NAACL-HLT, Minneapolis, MN, USA, 2019, pp. 4171–4186.
- [4] H. Gascon, C. Uriarte, and K. Rieck, "Exploring the use of text classification in the cyber threat analysis domain," in Proc. ACM Workshop on Artificial Intelligence and Security (AISec), 2017, pp. 13–23.
- [5] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter," arXiv preprint arXiv:1910.01108, 2019.
- [6] Y. Liu et al., "RoBERTa: A robustly optimized BERT pretraining approach," arXiv preprint arXiv:1907.11692, 2019.
- [7] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in Proc. NeurIPS, 2017, pp. 4765–4774.
- [8] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2016, pp. 785–794.
- [9] I. Jamal et al., "An improved transformer-based model for detecting phishing, spam and ham emails: A large language model approach," Security and Privacy, Wiley, 2024, doi: 10.1002/spy2.402.
- [10] A. Alhuzali, A. Alloqmani, M. Aljabri, and R. Alharbi, "In-depth analysis of phishing email detection: Evaluating the performance of machine learning and deep learning models across multiple datasets," 2025.